How the Go-based link checker works

Katsutoshi Seki | Published: November 21, 2025 | Source: https://sekika.github.io/2025/11/21/go-linkchecker/

To automate website link checking, you need to process many URLs efficiently while also avoiding excessive load on target servers. Go excels at concurrent processing using goroutines and channels, which allows link checking to be performed in parallel.

However, naive parallelization may send many requests to the same host in a short period of time, potentially overloading the server. The link checker introduced in this article implements a mechanism where **each host is assigned its own worker goroutine, and requests to that host are spaced out at a fixed interval**.

This article explains how this access control is implemented in linkchecker through two functions: FetchHTTP and RunWorkers. It is slightly different from the latest code.

1.FetchHTTP: Sending a GET request and checking the status

```
func FetchHTTP(link string, client *http.Client, userAgent string) error {
  req, err := http.NewRequest("GET", link, nil)
  if err != nil {
   return err
  }
  req.Header.Set("User-Agent", userAgent)
                              req.Header.Set("Accept", "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8")
  req.Header.Set("Accept-Language", "en-US,en;q=0.5")
  req.Header.Set("Accept-Encoding", "gzip, deflate")
  req.Header.Set("Connection", "keep-alive")
  req.Header.Set("Cache-Control", "max-age=0")
  req.Header.Set("Sec-Fetch-Dest", "document")
  resp, err := client.Do(req)

  if err != nil {
   return err
  }
  defer resp.Body.Close()
  if resp.StatusCode >= 400 {
   return fmt.Errorf("status %d", resp.StatusCode)
  }
  return nil
}
```

This function:

- Sends a GET request
- Sets the User-Agent
- Returns an error for HTTP status codes 400 or higher

In short, it performs a simple HTTP access and error check.

2.RunWorkers: Creating workers per host and processing links

Now we move on to the core of the implementation: the RunWorkers function.

```
func RunWorkers(
  links []string,
  baseURL string,
  timeoutSec int,
  waitSec int,
  userAgent string,
) {
```

2.1.1. Resolving relative URLs

Relative URLs are resolved here. You could also add filtering logic, such as excluding certain hosts.

```
base, _ := url.Parse(baseURL)
filteredLinks := []string{}

for _, link := range links {
  u, err := base.Parse(link)
  if err != nil {
   continue
  }
  filteredLinks = append(filteredLinks, u.String())
}
```

2.2.2. Preparing a map to store channels for each host

```
hostQueues := make(map[string]chan string)
var mu sync.Mutex
var wg sync.WaitGroup
```

Here we prepare a **map** where each key is a host and the value is the channel for that host.

- mu is a mutex used to safely access the map, preventing race conditions when multiple goroutines read/write concurrently.
- wg is a WaitGroup used to track active workers and wait for them all to finish.

2.3.3. Processing each link and creating workers per host

This is the important loop:

```
for _, link := range filteredLinks {
  u, _ := url.Parse(link)
  host := u.Host
```

For each link:

- Extract the host (e.g., example.com)
- Check whether a worker already exists for that host

2.3.1.3-1. Create a queue and worker if the host is new

```
  mu.Lock()
  if _, ok := hostQueues[host]; !ok {
   hostQueues[host] = make(chan string, 100)
```

```
   ch := hostQueues[host]
```

If this host is encountered for the first time:

- Create a new channel
- Launch a worker goroutine for that host

2.3.2.3-2. Launching a worker goroutine (one per host)

```
   wg.Add(1)
   go func(host string, ch chan string) {
    defer wg.Done()
    jar, _ := cookiejar.New(nil)
    client := &http.Client{
    Timeout: time.Duration(timeoutSec) * time.Second,
    Jar:  jar,
    }
    for l := range ch {
    err := FetchHTTP(l, client, userAgent)
    if err != nil {
      fmt.Printf("[NG] %s (%v)\n", l, err)
    } else {
      fmt.Printf("[OK] %s\n", l)
    }
    time.Sleep(time.Duration(waitSec) * time.Second)
    }
   }(host, ch)
  }
  mu.Unlock()
```

This worker processes only the URLs belonging to the assigned host.

for l := range ch continues until the channel is closed.

A fixed delay:

```
time.Sleep(waitSec)
```

ensures spacing between requests to the same host.

The worker creates a per-host HTTP client, complete with timeout and cookie jar:

- Timeout prevents hanging indefinitely
- Cookie jar allows handling cookies per host

2.3.3.3-3. Sending the URL to the host's channel

```
  hostQueues[host] <- link
}
```

Each link is pushed into the appropriate host-specific queue.

2.4.4. Closing all channels

```
for _, ch := range hostQueues {
  close(ch)
}
```

Closing a channel signals its worker to finish, allowing:

```
for l := range ch
```

to exit.

2.5.5. Waiting for all workers to finish

```
wg.Wait()
```

Without this, the program might exit while workers are still running.

wg works as follows:

1. Increase the counter before starting a worker (wg.Add(1))
2. Decrease the counter when the worker finishes (wg.Done())
3. Wait for all workers to exit (wg.Wait())

This ensures that the program only terminates after all link checks have completed.

3.Summary of the whole structure

1. Filter and normalize URL list
2. Iterate over each link
3. For each link:
   - Extract host
   - Create channel/worker if missing
   - Send link into host's queue
4. After the loop, close all channels
5. Wait for all workers to finish

4.Required imports and their usage

```
import (
  "fmt"
  "log"
  "net/http"
  "net/http/cookiejar"
  "net/url"
  "sync"
  "time"
)
```

- net/http — HTTP client access
- net/http/cookiejar — cookie handling
- net/url — resolving relative URLs
- time — controlling intervals and timeouts
- log — printing results
- sync — mutex and WaitGroup management
- fmt — formatting error messages